Multi-Client Server Patterns - Scaling Beyond One Connection

Theory, Edge Cases & Code Examples

- ✓ The Single-Client Problem
- ✓ Multi-Client Server Strategies
- ✓ Strategy 1: Fork-Based Server (Process Per Client)
- ✓ Strategy 2: Thread-Based Server (Thread Per Client)
- ✓ Strategy 3: Thread Pool Pattern

→ *Swipe to learn* →

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 The Single-Client Problem

In Topic 1, we built servers that handle one client at a time. This is like a restaurant with one table—customers must wait in line until the previous customer finishes eating. For most real-world applications, this is unacceptable.

```
while (true) {
    int client_fd = accept(server_fd, ...);  // Accept one client
    handle_client(client_fd);                // Blocks here until done
    close(client_fd);                        // Only then can we accept next client
}
```

Problem: While handling one client, all other clients are blocked. If `handle_client()` takes 10 seconds, the second client waits 10 seconds, the third waits 20 seconds, and so on.

In autonomous vehicle systems, imagine a cloud server that can only handle one vehicle's request at a time. If 1,000 vehicles need map updates simultaneously, the last vehicle waits over 2 hours!

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 Multi-Client Server Strategies

There are several approaches to handle multiple clients concurrently:

1. **Fork-based (Process per client)** - Create new process for each client
2. **Thread-based (Thread per client)** - Create new thread for each client
3. **Thread Pool** - Pre-create threads, assign clients to available threads
4. **I/O Multiplexing** - Single thread monitors multiple connections (Topics 3-5)

Each strategy has trade-offs in complexity, scalability, and resource usage.

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 Strategy 1: Fork-Based Server (Process Per Client)

Concept: When a client connects, fork() creates a child process to handle that client. The parent continues accepting new clients.

```
Parent Process                    Child Processes
     |                                  |
accept() —> fork() —> Child 1 —> handle_client() —> exit
     |                                  |
accept() —> fork() —> Child 2 —> handle_client() —> exit
     |                                  |
accept() —> fork() —> Child 3 —> handle_client() —> exit
```

How it works:
- Parent listens for connections
- On new client, fork() creates identical copy of process
- Child process handles client, then exits
- Parent immediately accepts next client (no blocking)

Advantages:
- **Simple**: Each child is independent, straightforward code
- **Isolation**: One client crash doesn't affect others (separate memory spaces)
- **Robust**: Parent can monitor and restart crashed children

Disadvantages:
- **Heavy**: Each process consumes significant memory (separate address space)
- **Slow creation**: fork() is expensive (copy memory, file descriptors)
- **Limited scalability**: OS limits on process count (~1000-10000)
- **IPC complexity**: Processes need shared memory/pipes to communicate

When to use:
- Small number of concurrent clients (< 100)
- Long-lived connections
- Need strong isolation between clients
- Security-critical applications (compromised child can't affect parent)

Real-world example: Traditional Apache web server (MPM prefork mode)

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 Strategy 2: Thread-Based Server (Thread Per Client)

Concept: When a client connects, create a new thread to handle that client. All threads share the same process memory.

```
Main Thread                          Worker Threads
     |                                     |
accept() ——> pthread_create() ——> Thread 1 ——> handle_client()
     |                                     |
accept() ——> pthread_create() ——> Thread 2 ——> handle_client()
     |                                     |
accept() ——> pthread_create() ——> Thread 3 ——> handle_client()
```

How it works:
- Main thread listens for connections
- On new client, pthread_create() or std::thread creates worker thread
- Worker thread handles client, then exits
- Main thread immediately accepts next client

Advantages:
- **Lighter than processes**: Threads share memory, lower overhead
- **Faster creation**: Thread creation faster than fork()
- **Easy communication**: Shared memory makes data sharing simple
- **Better scalability**: Can handle more clients than fork-based

Disadvantages:
- **Shared memory bugs**: Race conditions, deadlocks if not careful
- **No isolation**: One thread crash can crash entire process
- **Thread limit**: Still limited by OS (typically 1000-10000 threads)
- **Context switching**: Too many threads degrades performance

When to use:
- Moderate number of concurrent clients (100-1000)
- Need to share data between clients (e.g., chat server)
- Shorter connections
- Performance matters more than isolation

Real-world example: Many game servers, chat servers

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 Strategy 3: Thread Pool Pattern (1/2)

Concept: Pre-create a fixed number of worker threads. When client connects, assign to available thread from pool. Threads are reused, not created/destroyed per client.

```
Main Thread              Thread Pool                Connection Queue
     |                      |                             |
accept() ──> enqueue() ──> [Thread 1] ──> dequeue() ──> handle_client()
     |                      [Thread 2] ──> dequeue() ──> handle_client()
accept() ──> enqueue() ──> [Thread 3] ──> dequeue() ──> handle_client()
     |                      [Thread 4] ──> dequeue() ──> handle_client()
accept() ──> enqueue() ──>   ...
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 Strategy 3: Thread Pool Pattern (2/2)

How it works:
- Create fixed number of worker threads at startup (e.g., 10 threads)
- Threads wait on a queue for work
- When client connects, add connection to queue
- Available thread picks up connection from queue and handles it
- After handling, thread returns to pool (not destroyed)

Advantages:
- **No creation overhead**: Threads created once, reused many times
- **Bounded resources**: Fixed thread count prevents resource exhaustion
- **Better performance**: Optimal thread count for CPU cores (e.g., 8 threads for 8 cores)
- **Graceful degradation**: Queue buffers bursts, clients wait but aren't rejected

Disadvantages:
- **More complex**: Need thread-safe queue, synchronization
- **Queue management**: Must handle queue full scenarios
- **Tuning required**: Thread count affects performance (too few = bottleneck, too many = overhead)

When to use:
- High-throughput servers (thousands of short-lived connections)
- Known workload characteristics
- Production systems where predictable resource usage matters
- When clients can tolerate brief queuing

Optimal thread count: Typically number of CPU cores for CPU-bound work, or 2-4x CPU cores for I/O-bound work.

Real-world example: Web servers (nginx with thread pools), database connection pools

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 Strategy Comparison Table

Aspect	Fork-Based	Thread-Based	Thread Pool	I/O Multiplexing
Creation cost	High (fork)	Medium (thread create)	Low (reuse)	None
Memory usage	High (separate address space)	Medium (shared memory)	Low (fixed threads)	Very low (single thread)
Max clients	~100-1000	~1000-10000	Limited by queue	100,000+
Complexity	Simple	Moderate	High	Very high
Isolation	Strong	None	None	None
Performance	Lowest	Medium	High	Highest
Use case	Small scale, security	Medium scale	High throughput	Massive scale

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 Zombie Processes and SIGCHLD

When using fork(), child processes become **zombies** after exit if parent doesn't wait() for them.

Zombie process: Process that finished execution but still has entry in process table (waiting for parent to read exit status).

Problem:

```
// Parent keeps forking
while (true) {
    int pid = fork();
    if (pid == 0) {
        handle_client(client_fd);
        exit(0);  // Child exits, becomes zombie
    }
    // Parent never calls wait() - zombies accumulate!
}
```

Check zombies: `ps aux | grep defunct`

Solution 1: Install SIGCHLD handler to reap zombies

```
void sigchld_handler(int sig) {
    while (waitpid(-1, NULL, WNOHANG) > 0);  // Reap all dead children
}

signal(SIGCHLD, sigchld_handler);
```

Solution 2: Double fork trick (child orphans grandchild, init reaps it)

Why it matters: Process table has finite size. Too many zombies → can't create new processes (fork() fails with EAGAIN).

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 Thread Safety and Race Conditions

When multiple threads share data, **race conditions** occur when operations interleave unpredictably.

Example race condition:

```
int client_count = 0;  // Shared by all threads

void handle_client() {
    client_count++;  // ❌ Not atomic! Race condition
    // ...
}
```

Why it's a bug: `client_count++` is actually three operations:

1. Read client_count from memory → register
2. Increment register
3. Write register → memory

If two threads execute simultaneously:

```
Thread 1: Read client_count (0) → register
Thread 2: Read client_count (0) → register  // Both read 0!
Thread 1: Increment register (1)
Thread 2: Increment register (1)            // Both get 1!
Thread 1: Write (1) → client_count
Thread 2: Write (1) → client_count          // Final value = 1 (should be 2!)
```

Solution: Use mutex (mutual exclusion)

```
std::mutex client_mutex;
int client_count = 0;

void handle_client() {
    {
        std::lock_guard<std::mutex> lock(client_mutex);  // ✅ Protected
        client_count++;
    }  // Mutex automatically released
}
```

Critical section: Code that accesses shared data. Only one thread should execute critical section at a time.

Multi-Client Server Patterns - Scaling Beyond One Connection

Theory

📌 Why Multi-Client Servers Matter for Systems Engineers

Understanding concurrency patterns is fundamental because:

1. **Real-world servers are concurrent**: No production system handles one request at a time
2. **Performance vs complexity trade-off**: Must choose right pattern for scale requirements
3. **Resource management**: CPU, memory, and file descriptors are limited
4. **Debugging concurrent bugs**: Race conditions, deadlocks are common in production
5. **Interview relevance**: Concurrency questions are common for backend/systems roles

In autonomous vehicles:
- **Cloud gateway**: Thousands of vehicles connecting simultaneously
- **Map server**: Vehicles requesting HD map tiles (must handle burst traffic)
- **Telemetry collector**: Ingesting sensor data from entire fleet
- **OTA update server**: Rolling out software updates to vehicles efficiently

All require multi-client handling strategies.

Multi-Client Server Patterns - Scaling Beyond One Connection

Edge Cases

📌 1. Zombie Process Accumulation (1/2)

```
void start_server() {
    int server_fd = create_listening_socket(5000);

    while (true) {
        int client_fd = accept(server_fd, NULL, NULL);

        pid_t pid = fork();
        if (pid == 0) {
            // Child process
            close(server_fd);  // Child doesn't need listening socket
            handle_client(client_fd);
            close(client_fd);
            exit(0);  // Child exits cleanly
        }

        // Parent process
        close(client_fd);  // Parent doesn't need client socket
        // ❌ BUG: Never reaps child - zombie accumulates!
    }
}
```

After 100 clients connect:

```
$ ps aux | grep defunct
1001  12345  0.0  0.0     0     0  ?  Z  10:00  0:00 [server] <defunct>
1001  12346  0.0  0.0     0     0  ?  Z  10:00  0:00 [server] <defunct>
... 98 more zombies ...
```

Why zombies are bad:
- Consume process table entries (finite resource)
- Eventually fork() fails with EAGAIN
- Server can't accept new clients
- Reboot required to clear (zombies can't be killed with kill -9!)

Solution:

Multi-Client Server Patterns - Scaling Beyond One Connection

Edge Cases

📌 1. Zombie Process Accumulation (2/2)

```
#include <signal.h>
#include <sys/wait.h>

void sigchld_handler(int signum) {
    // Reap all dead children
    int saved_errno = errno;  // save errno
    while (waitpid(-1, NULL, WNOHANG) > 0);
    errno = saved_errno;  // restore errno
}

void start_server() {
    // Install SIGCHLD handler before forking
    struct sigaction sa;
    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;  // Restart interrupted system calls
    sigaction(SIGCHLD, &sa, NULL);

    // Now server code is safe
    while (true) {
        int client_fd = accept(server_fd, NULL, NULL);
        // ... fork and handle client
    }
}
```

WNOHANG flag: Non-blocking wait. Returns immediately if no child has exited.

SA_RESTART flag: Automatically restart system calls (accept, recv) interrupted by SIGCHLD.

Multi-Client Server Patterns - Scaling Beyond One Connection

Edge Cases

📌 2. File Descriptor Leaks in Fork (1/2)

```
while (true) {
    int client_fd = accept(server_fd, NULL, NULL);

    if (fork() == 0) {
        // Child: handle client
        handle_client(client_fd);
        close(client_fd);
        exit(0);
    }

    // Parent: ❌ FORGOT to close client_fd
    // Parent now has copy of client_fd that never gets closed!
}
```

What happens:
- fork() duplicates all file descriptors to child
- Both parent and child have reference to client_fd
- Reference count = 2
- Child closes client_fd (ref count = 1)
- Connection stays open because parent still has reference!

Symptoms:
- Connections don't close cleanly
- File descriptor leak in parent (eventually hits EMFILE)
- Client may see timeout instead of clean disconnect

Correct pattern:

```
while (true) {
    int client_fd = accept(server_fd, NULL, NULL);

    pid_t pid = fork();
    if (pid == 0) {
        // Child
        close(server_fd);  // Child doesn't need listening socket
        handle_client(client_fd);
        close(client_fd);
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Edge Cases

📌 2. File Descriptor Leaks in Fork (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        exit(0);
    }

    // Parent
    close(client_fd);  // ✅ Parent must close its copy
}
```

General rule: After fork(), close file descriptors you don't need in each process.

Multi-Client Server Patterns - Scaling Beyond One Connection

Edge Cases

📌 3. Thread Detachment and Memory Leaks

```
void start_server() {
    while (true) {
        int client_fd = accept(server_fd, NULL, NULL);

        std::thread t(handle_client, client_fd);
        // ❌ BUG: Thread not joined or detached - memory leak!
    }
}
```

Problem: std::thread destructor calls std::terminate() if thread is not joined or detached before destruction.

Two options:

Option 1: Detach thread (runs independently, resources auto-cleaned)

```
std::thread t(handle_client, client_fd);
t.detach();  // ✅ Thread continues independently
// Thread resources freed when function exits
```

Option 2: Join thread (wait for completion)

```
std::thread t(handle_client, client_fd);
t.join();  // ❌ Blocks until thread finishes - defeats purpose!
```

For multi-client servers, **detach is usually correct** because we don't want to wait.

Memory leak without detach:
- Each thread consumes stack memory (~8MB default)
- Thread handle stored in std::thread object
- Without join/detach, resources never freed
- 1000 clients = 8GB memory leaked!

Multi-Client Server Patterns - Scaling Beyond One Connection

Edge Cases

📌 4. Shared State Without Synchronization (1/2)

```
std::vector<int> active_clients;  // Shared by all threads

void handle_client(int client_fd) {
    active_clients.push_back(client_fd);  // ❌ Race condition!

    // Handle client...

    // Remove from list
    auto it = std::find(active_clients.begin(), active_clients.end(), client_fd);
    active_clients.erase(it);  // ❌ Race condition!
}
```

Problem: std::vector is NOT thread-safe. Multiple threads modifying simultaneously causes:
- **Corruption**: Internal vector state becomes invalid
- **Crashes**: Segmentation fault from corrupted pointers
- **Data loss**: Some insertions/deletions lost

Example corruption:

```
Thread 1: active_clients.push_back(5)
  - Checks capacity: size=10, capacity=10, needs resize
  - Allocates new array (capacity=20)
Thread 2: active_clients.push_back(7)  // Happens during Thread 1's resize!
  - Writes to old array (about to be freed)
Thread 1: Copies elements to new array
  - Frees old array
Thread 2's write is now in freed memory - CORRUPTION!
```

Solution:

```
std::mutex clients_mutex;
std::vector<int> active_clients;

void handle_client(int client_fd) {
    {
        std::lock_guard<std::mutex> lock(clients_mutex);
        active_clients.push_back(client_fd);  // ✅ Protected
    }
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Edge Cases

📌 4. Shared State Without Synchronization (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    // Handle client...

    {
        std::lock_guard<std::mutex> lock(clients_mutex);
        auto it = std::find(active_clients.begin(), active_clients.end(), client_fd);
        active_clients.erase(it);  // ✅ Protected
    }
}
```

Critical sections should be small: Lock only when accessing shared data, release ASAP.

Multi-Client Server Patterns - Scaling Beyond One Connection

Edge Cases

📌 5. Thread Pool Queue Full (1/2)

```
class ThreadPool {
    std::queue<int> client_queue;  // Bounded queue
    static constexpr size_t MAX_QUEUE_SIZE = 100;

public:
    void add_client(int client_fd) {
        std::lock_guard<std::mutex> lock(queue_mutex);

        if (client_queue.size() >= MAX_QUEUE_SIZE) {
            // ❌ Queue full! What to do?
            // Option 1: Block accept() (bad - main thread hangs)
            // Option 2: Drop client (bad - silent failure)
            // Option 3: Reject gracefully (good!)
        }

        client_queue.push(client_fd);
    }
};
```

Problem: Under heavy load, queue fills up. Must handle gracefully.

Wrong solution 1: Block forever

```
while (client_queue.size() >= MAX_QUEUE_SIZE) {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
}
```

Main thread blocks, server stops accepting, clients timeout.

Wrong solution 2: Drop silently

```
if (client_queue.size() >= MAX_QUEUE_SIZE) {
    close(client_fd);  // Drop connection
    return;
}
```

Client thinks connection succeeded, then times out waiting for response.

Multi-Client Server Patterns - Scaling Beyond One Connection

Edge Cases

📌 5. Thread Pool Queue Full (2/2)

Correct solution: Reject with error message

```
if (client_queue.size() >= MAX_QUEUE_SIZE) {
    const char* error = "HTTP/1.1 503 Service Unavailable\r\n\r\nServer overloaded\r\n";
    send(client_fd, error, strlen(error), MSG_NOSIGNAL);
    close(client_fd);
    return;
}
```

Client receives explicit error, can retry or failover.

Even better: Backpressure

```
if (client_queue.size() > MAX_QUEUE_SIZE * 0.9) {
    // 90% full - send warning header
    const char* warning = "X-Queue-Depth: High\r\n";
    send(client_fd, warning, strlen(warning), MSG_NOSIGNAL);
}
```

Production strategy: Combine queue limits + metrics + auto-scaling.

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 1. Fork-Based Echo Server (1/4)

Key points:
- SIGCHLD handler reaps zombies automatically
- Parent closes client_fd, child closes server_fd (no leaks)
- SA_RESTART flag ensures accept() resumes after signal
- Each child is isolated process

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/socket.h>
#include <arpa/inet.h>

// SIGCHLD handler to reap zombie processes
void sigchld_handler(int signum) {
    int saved_errno = errno;
    // Reap all dead children without blocking
    while (waitpid(-1, NULL, WNOHANG) > 0);
    errno = saved_errno;
}

void handle_client(int client_fd) {
    char buffer[1024];

    while (true) {
        ssize_t bytes = recv(client_fd, buffer, sizeof(buffer) - 1, 0);

        if (bytes <= 0) {
            if (bytes < 0) perror("recv");
            break;  // Connection closed or error
        }

        buffer[bytes] = '\0';
        std::cout << "Child PID " << getpid() << " received: " << buffer << "\n";

        // Echo back
        send(client_fd, buffer, bytes, MSG_NOSIGNAL);
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 1. Fork-Based Echo Server (2/4)

```
// ...continued from previous slide
    }

    std::cout << "Child PID " << getpid() << " closing connection\n";
}

int main() {
    // Install SIGCHLD handler
    struct sigaction sa;
    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;  // Restart interrupted syscalls
    if (sigaction(SIGCHLD, &sa, NULL) < 0) {
        perror("sigaction");
        return 1;
    }

    // Create listening socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(5000);

    if (bind(server_fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind");
        close(server_fd);
        return 1;
    }

    if (listen(server_fd, 128) < 0) {
        perror("listen");
        close(server_fd);
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 1. Fork-Based Echo Server (3/4)

```
// ...continued from previous slide
        return 1;
    }

    std::cout << "Fork-based server listening on port 5000...\n";

    while (true) {
        sockaddr_in client_addr{};
        socklen_t client_len = sizeof(client_addr);
        int client_fd = accept(server_fd, (sockaddr*)&client_addr, &client_len);

        if (client_fd < 0) {
            if (errno == EINTR) continue;  // Interrupted by SIGCHLD, retry
            perror("accept");
            continue;
        }

        std::cout << "Parent: Accepted connection, forking...\n";

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            close(client_fd);
            continue;
        }

        if (pid == 0) {
            // Child process
            close(server_fd);  // Child doesn't need listening socket
            handle_client(client_fd);
            close(client_fd);
            exit(0);  // Child exits cleanly
        }

        // Parent process
        close(client_fd);  // Parent doesn't need client socket
        std::cout << "Parent: Forked child PID " << pid << "\n";
    }

    close(server_fd);
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 1. Fork-Based Echo Server (4/4)

[Code continues across slides]

```
// ...continued from previous slide
    return 0;
}
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 2. Thread-Based Echo Server (1/3)

Key points:
- std::atomic for thread-safe counter (no mutex needed for simple increment)
- t.detach() allows thread to run independently
- Lighter than fork (shared memory)
- No zombie processes to worry about

```
#include <iostream>
#include <thread>
#include <atomic>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>

std::atomic<int> active_threads{0};  // Thread-safe counter

void handle_client(int client_fd) {
    active_threads++;
    std::cout << "Thread " << std::this_thread::get_id()
              << " handling client (active: " << active_threads << ")\n";

    char buffer[1024];

    while (true) {
        ssize_t bytes = recv(client_fd, buffer, sizeof(buffer) - 1, 0);

        if (bytes <= 0) {
            if (bytes < 0) perror("recv");
            break;
        }

        buffer[bytes] = '\0';
        std::cout << "Thread " << std::this_thread::get_id()
                  << " received: " << buffer << "\n";

        send(client_fd, buffer, bytes, MSG_NOSIGNAL);
    }

// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 2. Thread-Based Echo Server (2/3)

```
// ...continued from previous slide
    close(client_fd);
    active_threads--;
    std::cout << "Thread " << std::this_thread::get_id()
              << " closing (active: " << active_threads << ")\n";
}

int main() {
    // Create listening socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(5000);

    if (bind(server_fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind");
        close(server_fd);
        return 1;
    }

    if (listen(server_fd, 128) < 0) {
        perror("listen");
        close(server_fd);
        return 1;
    }

    std::cout << "Thread-based server listening on port 5000...\n";

    while (true) {
        int client_fd = accept(server_fd, NULL, NULL);

        if (client_fd < 0) {
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 2. Thread-Based Echo Server (3/3)

[Code continues across slides]

```
// ...continued from previous slide
            perror("accept");
            continue;
        }

        std::cout << "Main: Accepted connection, creating thread...\n";

        // Create detached thread to handle client
        std::thread t(handle_client, client_fd);
        t.detach();  // Thread runs independently
    }

    close(server_fd);
    return 0;
}
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 3. Thread Pool Server (Production Pattern) (1/5)

Key points:
- Fixed number of worker threads (optimal = CPU cores)
- std::condition_variable for efficient waiting (no busy-waiting)
- Queue bounded to prevent memory exhaustion
- Rejects clients gracefully when overloaded
- Threads reused, not created per client
- Production-ready pattern

```
#include <iostream>
#include <thread>
#include <queue>
#include <mutex>
#include <condition_variable>
#include <vector>
#include <atomic>
#include <cstring>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>

class ThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<int> client_queue;
    std::mutex queue_mutex;
    std::condition_variable condition;
    std::atomic<bool> stop{false};
    std::atomic<int> active_clients{0};

    void worker_thread() {
        while (true) {
            int client_fd;

            {
                std::unique_lock<std::mutex> lock(queue_mutex);

                // Wait for work or stop signal
                condition.wait(lock, [this] {
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 3. Thread Pool Server (Production Pattern) (2/5)

```
// ...continued from previous slide
                    return stop || !client_queue.empty();
                });

                if (stop && client_queue.empty()) {
                    return;  // Thread exits
                }

                client_fd = client_queue.front();
                client_queue.pop();
            }

            // Handle client outside lock
            handle_client(client_fd);
        }
    }

    void handle_client(int client_fd) {
        active_clients++;
        std::cout << "Thread " << std::this_thread::get_id()
                  << " handling client (active: " << active_clients << ")\n";

        char buffer[1024];

        while (true) {
            ssize_t bytes = recv(client_fd, buffer, sizeof(buffer) - 1, 0);

            if (bytes <= 0) {
                if (bytes < 0) perror("recv");
                break;
            }

            buffer[bytes] = '\0';
            send(client_fd, buffer, bytes, MSG_NOSIGNAL);
        }

        close(client_fd);
        active_clients--;
        std::cout << "Thread " << std::this_thread::get_id()
                  << " finished (active: " << active_clients << ")\n";
    }
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 3. Thread Pool Server (Production Pattern) (3/5)

```
// ...continued from previous slide

public:
    ThreadPool(size_t num_threads) {
        for (size_t i = 0; i < num_threads; i++) {
            workers.emplace_back([this] { worker_thread(); });
        }
        std::cout << "Thread pool created with " << num_threads << " workers\n";
    }

    ~ThreadPool() {
        stop = true;
        condition.notify_all();

        for (std::thread& worker : workers) {
            if (worker.joinable()) {
                worker.join();
            }
        }
    }

    bool add_client(int client_fd) {
        {
            std::lock_guard<std::mutex> lock(queue_mutex);

            // Check queue size limit
            if (client_queue.size() >= 100) {
                std::cerr << "Queue full, rejecting client\n";
                const char* error = "HTTP/1.1 503 Service Unavailable\r\n\r\n";
                send(client_fd, error, strlen(error), MSG_NOSIGNAL);
                close(client_fd);
                return false;
            }

            client_queue.push(client_fd);
        }

        condition.notify_one();  // Wake up one worker
        return true;
    }

// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 3. Thread Pool Server (Production Pattern) (4/5)

```
// ...continued from previous slide
    size_t queue_size() const {
        std::lock_guard<std::mutex> lock(queue_mutex);
        return client_queue.size();
    }

    int get_active_clients() const {
        return active_clients;
    }
};

int main() {
    // Create thread pool with optimal size (e.g., CPU cores)
    size_t num_threads = std::thread::hardware_concurrency();
    if (num_threads == 0) num_threads = 4;  // Fallback
    ThreadPool pool(num_threads);

    // Create listening socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(5000);

    if (bind(server_fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind");
        close(server_fd);
        return 1;
    }

    if (listen(server_fd, 128) < 0) {
        perror("listen");
        close(server_fd);
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 3. Thread Pool Server (Production Pattern) (5/5)

[Code continues across slides]

```
// ...continued from previous slide
        return 1;
    }

    std::cout << "Thread pool server listening on port 5000...\n";
    std::cout << "Using " << num_threads << " worker threads\n";

    while (true) {
        int client_fd = accept(server_fd, NULL, NULL);

        if (client_fd < 0) {
            perror("accept");
            continue;
        }

        std::cout << "Accepted connection, adding to pool (queue: "
                  << pool.queue_size() << ", active: "
                  << pool.get_active_clients() << ")\n";

        pool.add_client(client_fd);
    }

    close(server_fd);
    return 0;
}
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 4. Comparing Performance (Benchmark) (1/3)

Typical output:

Observations:
- Thread creation ~2-3x faster than fork
- Performance gap widens with more clients
- Fork becomes impractical beyond ~500 clients
- Thread pool would be even faster (no creation overhead)

```
#include <iostream>
#include <chrono>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <sys/wait.h>
#include <thread>

// Simulate work (e.g., processing request)
void simulate_work() {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
}

// Fork-based: Handle N clients
void benchmark_fork(int num_clients) {
    auto start = std::chrono::steady_clock::now();

    for (int i = 0; i < num_clients; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            // Child
            simulate_work();
            exit(0);
        }
    }

    // Parent waits for all children
    for (int i = 0; i < num_clients; i++) {
        wait(NULL);
    }

// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 4. Comparing Performance (Benchmark) (2/3)

```
// ...continued from previous slide
    auto end = std::chrono::steady_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
    std::cout << "Fork-based: " << num_clients << " clients in "
              << duration.count() << "ms\n";
}

// Thread-based: Handle N clients
void benchmark_threads(int num_clients) {
    auto start = std::chrono::steady_clock::now();

    std::vector<std::thread> threads;
    for (int i = 0; i < num_clients; i++) {
        threads.emplace_back(simulate_work);
    }

    for (std::thread& t : threads) {
        t.join();
    }

    auto end = std::chrono::steady_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
    std::cout << "Thread-based: " << num_clients << " clients in "
              << duration.count() << "ms\n";
}

int main() {
    std::cout << "Benchmarking server strategies:\n\n";

    std::cout << "Small scale (10 clients):\n";
    benchmark_fork(10);
    benchmark_threads(10);

    std::cout << "\nMedium scale (100 clients):\n";
    benchmark_fork(100);
    benchmark_threads(100);

    // Note: Don't run fork with 1000 clients - will exhaust resources!
    std::cout << "\nLarge scale (1000 clients):\n";
    std::cout << "Fork-based: Skipped (resource intensive)\n";
    benchmark_threads(1000);
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 4. Comparing Performance (Benchmark) (3/3)

[Code continues across slides]

```
// ...continued from previous slide

    return 0;
}
```

```
['Small scale (10 clients):\nFork-based: 10 clients in 250ms\nThread-based: 10 clients in 150ms\n\nMedium scale (100 cl
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 5. Shared State with Mutex Protection (1/3)

Key points:
- Shared std::map protected by mutex
- Lock held only during critical sections (map access)
- Demonstrates practical use case (chat server)
- All modifications to shared state are synchronized

```
#include <iostream>
#include <thread>
#include <mutex>
#include <map>
#include <string>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>

// Global client registry (shared by all threads)
std::mutex registry_mutex;
std::map<int, std::string> client_registry;  // fd -> username

void register_client(int client_fd, const std::string& username) {
    std::lock_guard<std::mutex> lock(registry_mutex);
    client_registry[client_fd] = username;
    std::cout << "Registered: " << username << " (fd=" << client_fd << ")\n";
}

void unregister_client(int client_fd) {
    std::lock_guard<std::mutex> lock(registry_mutex);
    auto it = client_registry.find(client_fd);
    if (it != client_registry.end()) {
        std::cout << "Unregistered: " << it->second << " (fd=" << client_fd << ")\n";
        client_registry.erase(it);
    }
}

void broadcast_message(int sender_fd, const std::string& message) {
    std::lock_guard<std::mutex> lock(registry_mutex);

    std::string sender_name = "Unknown";
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 5. Shared State with Mutex Protection (2/3)

```
// ...continued from previous slide
    auto it = client_registry.find(sender_fd);
    if (it != client_registry.end()) {
        sender_name = it->second;
    }

    std::string full_message = sender_name + ": " + message;

    // Send to all clients except sender
    for (const auto& [fd, username] : client_registry) {
        if (fd != sender_fd) {
            send(fd, full_message.c_str(), full_message.size(), MSG_NOSIGNAL);
        }
    }
}

void handle_client(int client_fd) {
    // Read username
    char buffer[256];
    ssize_t bytes = recv(client_fd, buffer, sizeof(buffer) - 1, 0);
    if (bytes <= 0) {
        close(client_fd);
        return;
    }

    buffer[bytes] = '\0';
    std::string username(buffer);

    // Register client
    register_client(client_fd, username);

    // Chat loop
    while (true) {
        bytes = recv(client_fd, buffer, sizeof(buffer) - 1, 0);

        if (bytes <= 0) {
            break;  // Connection closed
        }

        buffer[bytes] = '\0';
        std::string message(buffer);
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 5. Shared State with Mutex Protection (3/3)

[Code continues across slides]

```
// ...continued from previous slide

        // Broadcast to all other clients
        broadcast_message(client_fd, message);
    }

    // Cleanup
    unregister_client(client_fd);
    close(client_fd);
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(5000);

    bind(server_fd, (sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 128);

    std::cout << "Chat server listening on port 5000...\n";

    while (true) {
        int client_fd = accept(server_fd, NULL, NULL);
        if (client_fd < 0) continue;

        std::thread t(handle_client, client_fd);
        t.detach();
    }

    return 0;
}
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 6. Graceful Shutdown (1/4)

Key points:
- Signal handlers set atomic flag
- Periodic timeout checks allow clean exit
- Sends goodbye message to clients
- Waits for all threads to finish (no abrupt termination)
- Production servers should implement graceful shutdown

```
#include <iostream>
#include <thread>
#include <atomic>
#include <vector>
#include <signal.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>

std::atomic<bool> shutdown_requested{false};
std::vector<int> active_connections;
std::mutex connections_mutex;

void signal_handler(int signum) {
    std::cout << "\nShutdown signal received...\n";
    shutdown_requested = true;
}

void handle_client(int client_fd) {
    // Add to active connections
    {
        std::lock_guard<std::mutex> lock(connections_mutex);
        active_connections.push_back(client_fd);
    }

    char buffer[1024];

    while (!shutdown_requested) {
        // Use timeout to check shutdown_requested periodically
        struct timeval timeout;
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 6. Graceful Shutdown (2/4)

```
// ...continued from previous slide
        timeout.tv_sec = 1;
        timeout.tv_usec = 0;
        setsockopt(client_fd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));

        ssize_t bytes = recv(client_fd, buffer, sizeof(buffer) - 1, 0);

        if (bytes > 0) {
            buffer[bytes] = '\0';
            send(client_fd, buffer, bytes, MSG_NOSIGNAL);
        } else if (bytes == 0) {
            break;  // Connection closed
        } else if (errno != EAGAIN && errno != EWOULDBLOCK) {
            break;  // Real error
        }
        // EAGAIN/EWOULDBLOCK = timeout, check shutdown_requested
    }

    // Send goodbye message
    const char* goodbye = "Server shutting down, goodbye!\n";
    send(client_fd, goodbye, strlen(goodbye), MSG_NOSIGNAL);

    close(client_fd);

    // Remove from active connections
    {
        std::lock_guard<std::mutex> lock(connections_mutex);
        auto it = std::find(active_connections.begin(), active_connections.end(), client_fd);
        if (it != active_connections.end()) {
            active_connections.erase(it);
        }
    }

    std::cout << "Client disconnected gracefully\n";
}

int main() {
    // Install signal handlers
    signal(SIGINT, signal_handler);   // Ctrl+C
    signal(SIGTERM, signal_handler);  // kill command

// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 6. Graceful Shutdown (3/4)

```
// ...continued from previous slide
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(5000);

    bind(server_fd, (sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 128);

    // Make accept() non-blocking to check shutdown_requested
    struct timeval timeout;
    timeout.tv_sec = 1;
    timeout.tv_usec = 0;
    setsockopt(server_fd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));

    std::cout << "Server with graceful shutdown listening on port 5000...\n";
    std::cout << "Press Ctrl+C to shutdown gracefully\n";

    std::vector<std::thread> threads;

    while (!shutdown_requested) {
        int client_fd = accept(server_fd, NULL, NULL);

        if (client_fd < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                continue;  // Timeout, check shutdown_requested
            }
            perror("accept");
            continue;
        }

        std::thread t(handle_client, client_fd);
        threads.push_back(std::move(t));
    }

    std::cout << "No longer accepting new connections...\n";
    close(server_fd);
// ...continues on next slide
```

Multi-Client Server Patterns - Scaling Beyond One Connection

Code Examples

📌 6. Graceful Shutdown (4/4)

[Code continues across slides]

```
// ...continued from previous slide

    std::cout << "Waiting for " << threads.size() << " active clients to finish...\n";

    // Wait for all client threads to finish
    for (std::thread& t : threads) {
        if (t.joinable()) {
            t.join();
        }
    }

    std::cout << "All clients disconnected. Server shutdown complete.\n";
    return 0;
}
```

What You Learned

Key Takeaways

- ✓ The Single-Client Problem
- ✓ Multi-Client Server Strategies
- ✓ Strategy 1: Fork-Based Server (Process Per Client)
- ✓ Strategy 2: Thread-Based Server (Thread Per Client)

→ *Swipe to learn* →